25

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

파이프: 변환과 검증

제2부 NestJS 핵심 · 6장 요청 파이프라인

핸들러가 받는 값을 바꾸고 거르는 관문



© 한상훈 NestJS 백엔드 마스터

숫자 제목

curl

```
$ curl -X POST localhost:4025/concerts -d '{"title":12345,"venue":"x"}'
{"id":2,"title":12345,"venue":"x"} [201]
```

→ CreateConcertDto의 `title: string`

→ 타입 표시는 빌드에서 지워짐, 실행 때 검사 없음

© 한상훈 NestJS 백엔드 마스터

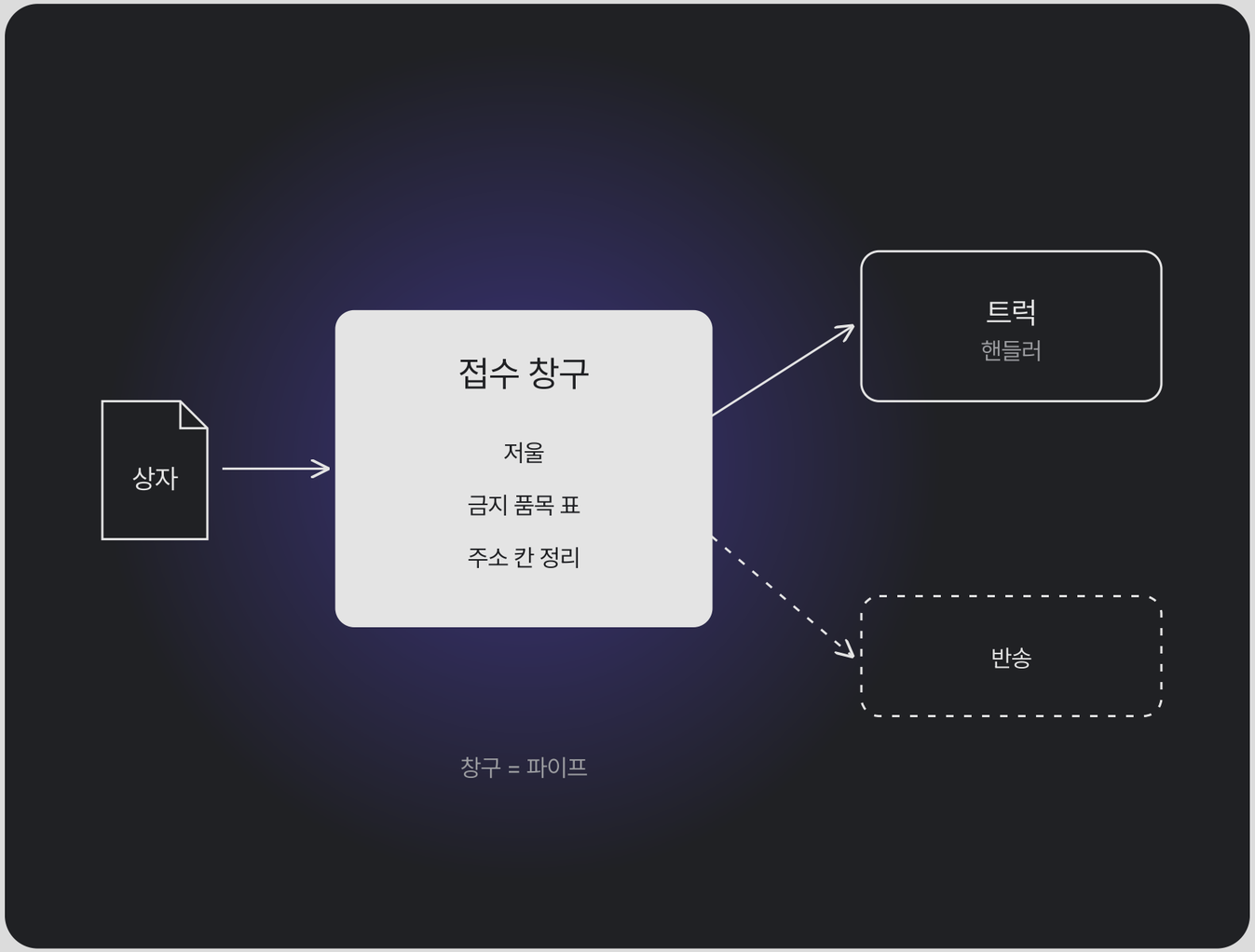
택배 접수 창구

→ 무게 측정, 금지 품목 확인

→ 주소 칸 정리(우편번호 형식 맞춤)

→ 통과한 상자만 트럭에 실림

→ 트럭 기사: 상자 안을 다시 확인하지 않음



© 한상훈 NestJS 백엔드 마스터

파이프

→ 핸들러 인자 하나를 받아서 바꾼 값을 넘기거나 예외를 던지는 클래스

→ `PipeTransform` 구현, `transform(value, metadata)` 하나

→ 변환: 문자열 `"7"` → 숫자 `7`

→ 검증: 형식이 틀리면 400

→ 실행 시점: 가드·인터셉터 앞부분 뒤, 핸들러 직전

© 한상훈 NestJS 백엔드 마스터

이미 쓴 파이프

```
@Get(':id')
findOne(@Param('id', ParseIntPipe) id: number) {
  return this.performancesService.findOne(id);
}
```

→ `ParseIntPipe`: 문자열을 정수로, 아니면 400

→ 내장 파이프: ParseIntPipe, ParseFloatPipe, ParseBoolPipe, ParseUUIDPipe, ParseEnumPipe, DefaultValuePipe, ValidationPipe 등

© 한상훈 NestJS 백엔드 마스터

소문자 좌석 번호

curl

```
$ curl -X POST localhost:4025/performances/1/seats/a-1-5/hold
{"type":"about:blank","title":"Not Found","status":404,
 "detail":"좌석 a-1-5이 없습니다", …} [404]
```

→ 좌석 번호: `A-1-5` (대문자)

→ 가정: 매표소 단말기 하나가 소문자로 보냄 → 모든 좌석이 "없는 좌석"

© 한상훈 NestJS 백엔드 마스터

SeatIdPipe

seat-id.pipe.ts

```ts
// 좌석 번호를 대문자로 맞추고 형식(구역-열-번호)을 검사한다. 예: a-1-5 → A-1-5
@Injectable()
export class SeatIdPipe implements PipeTransform<string, string> {
  transform(value: string, metadata: ArgumentMetadata): string {
    const seatId = value.trim().toUpperCase();
    if (!/^[A-Z]-\d{1,2}-\d{1,2}$/.test(seatId)) {
      throw new BadRequestException(`${metadata.data} 형식이 아닙니다: ${value} (예: A-3-15)`);
    }
    return seatId;
  }
}
```

```ts
hold(@Param('performanceId', ParseIntPipe) performanceId: number,
     @Param('seatId', SeatIdPipe) seatId: string) { … }
```

© 한상훈 NestJS 백엔드 마스터

바뀐 값과 거절

curl

```
$ curl -X POST localhost:4025/performances/1/seats/a-1-5/hold
{"id":"A-1-5","label":"A구역 1열 5번","status":"HELD"} [201]

$ curl -X POST localhost:4025/performances/1/seats/A15/hold
{"type":"about:blank","title":"Bad Request","status":400,
 "detail":"seatId 형식이 아닙니다: A15 (예: A-3-15)", …} [400]
```

© 한상훈 NestJS 백엔드 마스터

ArgumentMetadata

필드	뜻	SeatIdPipe에서
`type`	인자 출처: `'param'`, `'query'`, `'body'`, `'custom'`	`'param'`
`data`	데코레이터에 적은 이름	`'seatId'`
`metatype`	인자에 적은 타입의 클래스	`String`

→ `@Param('seatId', SeatIdPipe)`: 클래스를 넘김 → Nest가 생성, 주입 가능

→ `@Param('seatId', new SeatIdPipe())`: 직접 만든 객체, 주입 없음

→ 400(요청을 잘못 만듦)과 404(좌석이 없음): 고칠 곳이 다름

© 한상훈 NestJS 백엔드 마스터

ValidationPipe 켜기

터미널

```
$ npm install --os=win32 class-validator class-transformer
```

```
// main.ts
app.useGlobalFilters(new ProblemDetailsFilter());
app.useGlobalPipes(new ValidationPipe());
```

→ 전역 파이프: 모든 핸들러의 모든 인자

→ 핸들러마다 `@UsePipes`: 새 핸들러에서 빠뜨리면 검증 없는 API

→ 순서: 전역 파이프 → 인자에 건 파이프(SeatIdPipe)

데이터 블록: class-validator 0.15.1, class-transformer 0.5.1 (2026-10-26), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

CreateConcertDto

create-concert.dto.ts

```ts
import { IsNotEmpty, IsString, MaxLength } from 'class-validator';

export class CreateConcertDto {
  @IsString()
  @IsNotEmpty()
  @MaxLength(100)
  title: string;

  @IsString()
  @IsNotEmpty()
  @MaxLength(100)
  venue: string;
}
```

© 한상훈 NestJS 백엔드 마스터

errors 배열

curl

```
$ curl -X POST localhost:4025/concerts -d '{"title":12345,"venue":""}'
{"type":"about:blank","title":"Bad Request","status":400,
 "errors":["title must be shorter than or equal to 100 characters",
           "title must be a string",
           "venue should not be empty"],
 "instance":"/concerts"} [400]
```

→ 걸린 규칙 전부를 한 번에

→ Problem Details 필터: 검증 실패 메시지 배열 → `errors` 확장 필드

→ 기본 문구는 영어: 고객 화면에 띄우려면 `message` 옵션으로 한국어 (예: seatId 규칙)

© 한상훈 NestJS 백엔드 마스터

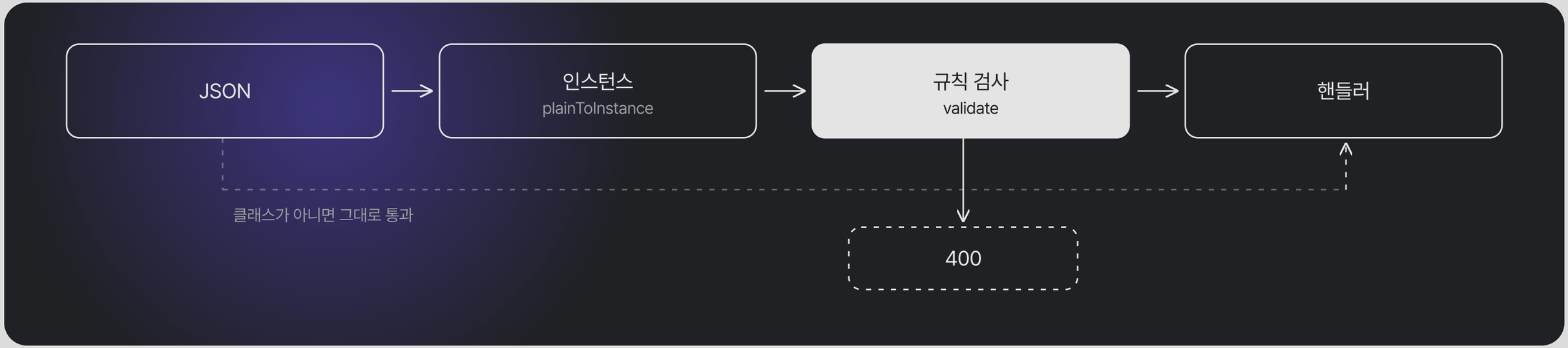
ValidationPipe가 하는 일

→ 1. `metatype` 확인: 클래스가 아니면(String, Number, Object 등) 그대로 통과

→ 2. `plainToInstance`: JSON 객체 → DTO 클래스 인스턴스 (class-transformer)

→ 3. `validate`: 클래스에 붙은 규칙 검사 (class-validator)

→ 4. 실패 → `BadRequestException`(메시지 배열), 성공 → 값 전달



© 한상훈 NestJS 백엔드 마스터

인라인 타입의 예매 본문

reservations.controller.ts

```ts
@Post()
reserve(
  @Body() body: { seatId: string; amount: number; buyer: string; channel?: string },
) {
  return this.reservations.reserve(body.seatId, body.amount, body.buyer, body.channel);
}
```

→ 인자 타입: 객체 리터럴 타입 → 실행 때 `metatype`은 Object

© 한상훈 NestJS 백엔드 마스터

승인 번호 fake-abc

curl · 서버 로그

```
$ curl -X POST localhost:4025/reservations \
    -d '{"seatId":"A-1-1","amount":"abc","buyer":"010-1234-5678"}'
{"id":1,"seatId":"A-1-1","approvalId":"fake-abc","amount":"abc",
 "message":"[문자 → 010-1234-5678] A-1-1 예매가 확정되었습니다"} [201]

[예매] A-1-1 결제 완료
```

→ 전역 ValidationPipe가 켜진 상태

→ 금액 "abc"가 결제사 호출까지 전달

© 한상훈 NestJS 백엔드 마스터

CreateReservationDto

create-reservation.dto.ts

```ts
export class CreateReservationDto {
  @Matches(/^[A-Z]-\d{1,2}-\d{1,2}$/, { message: 'seatId는 A-3-15 형식이어야 합니다' })
  seatId: string;

  @IsInt()
  @Min(1)
  amount: number;

  @IsString()
  @IsNotEmpty()
  buyer: string;

  @IsOptional()
  @IsIn(['sms', 'email', 'push'])
  channel?: string;
}
```

→ reserve(@Body() body: CreateReservationDto)

© 한상훈 NestJS 백엔드 마스터

같은 요청, 400

curl

```
$ curl -X POST localhost:4025/reservations \
    -d '{"seatId":"A-1-1","amount":"abc","buyer":"010-1234-5678"}'
{"type":"about:blank","title":"Bad Request","status":400,
 "errors":["amount must not be less than 1","amount must be an integer number"], …}

$ curl -X POST localhost:4025/reservations \
    -d '{"seatId":"A-1-1","amount":110000,"buyer":"010-1234-5678"}'
{"id":1,"seatId":"A-1-1","approvalId":"fake-110000","amount":110000, …} [201]
```

© 한상훈 NestJS 백엔드 마스터

좌석 목록 쿼리

```
export class ListSeatsQuery {
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  @Max(40)
  row?: number;

  @IsOptional()
  @IsIn(['AVAILABLE', 'HELD', 'SOLD'])
  status?: SeatStatus;
}
```

```
findAll(@Param('performanceId', ParseIntPipe) performanceId: number, @Query() query: ListSeatsQuery) {
  return this.seatsService.findAll(performanceId).filter(
    (s) => (query.row === undefined || s.row === query.row) && (query.status === undefined || s.status === query.status),
  );
}
```

© 한상훈 NestJS 백엔드 마스터

row=3이 0개

curl

```
$ curl "localhost:4025/performances/1/seats?row=3"
배열 0개

$ curl "localhost:4025/performances/1/seats?row=abc"
{"type":"about:blank","title":"Bad Request","status":400,
 "errors":["row must not be greater than 40","row must not be less than 1",
           "row must be an integer number"], …}
```

→ 검사는 통과, 핸들러가 받은 `query.row`: 문자열 "3"

→ `s.row === "3"` → 늘 false

© 한상훈 NestJS 백엔드 마스터

transform: true

```
app.useGlobalPipes(new ValidationPipe({ transform: true }));
```

curl

```
$ curl "localhost:4025/performances/1/seats?row=3"
배열 20개
$ curl "localhost:4025/performances/1/seats?row=3&status=AVAILABLE"
배열 20개
```

→ `transform`: 검사에 쓴 인스턴스(바뀐 값)를 핸들러에 전달

→ class-transformer: 평범한 객체를 클래스 인스턴스로, `@Type`으로 필드 타입 변환

© 한상훈 NestJS 백엔드 마스터

암묵 변환의 "false"

```
new ValidationPipe({ transform: true, transformOptions: { enableImplicitConversion: true } })
// ListSeatsQuery에 @IsOptional() @IsBoolean() aisle?: boolean;
```

○ ○ ○ curl · 서버 로그

```
$ curl "localhost:4025/performances/1/seats?row=3&aisle=false"
query ListSeatsQuery { row: 3, aisle: true, status: undefined }
```

→ 문자열 `"false"` → `Boolean("false")` → true

→ 본문 JSON의 `"1"`(문자열)은 `@Type` 없이 숫자로 바뀌지 않음

© 한상훈 NestJS 백엔드 마스터

whitelist와 forbidNonWhitelisted

```
app.useGlobalPipes(new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true }));
```

○ ○ ○ curl

```
$ curl -X POST localhost:4025/concerts -d '{"title":"Winter","venue":"Hall","settlementRate":0}'
{"type":"about:blank","title":"Bad Request","status":400,
 "errors":["property settlementRate should not exist"], …}

$ curl -X POST localhost:4025/reservations -d '{…,"discount":100}'
{"…","status":400,"errors":["property discount should not exist"], …}
```

→ `whitelist`: 규칙이 없는 필드를 지움

→ `forbidNonWhitelisted`: 지우는 대신 400

© 한상훈 NestJS 백엔드 마스터

PartialType이 가져오는 것

```
export class UpdateConcertDto extends PartialType(CreateConcertDto) {}
```

curl

```
$ curl -X PATCH localhost:4025/concerts/1 -d '{"title":"가가가…(300자)"}'
{"type":"about:blank","title":"Bad Request","status":400,
 "errors":["title must be shorter than or equal to 100 characters"], …}
```

→ 필드를 전부 선택으로 + 검증 규칙은 그대로

→ 복사한 수정 DTO: 등록 DTO 규칙을 고쳐도 따라오지 않음

© 한상훈 NestJS 백엔드 마스터

OmitType: 회차의 공연은 고정

```
export class CreatePerformanceDto {
  @IsInt() @Min(1) concertId: number;
  @IsISO8601() startsAt: string;
}

// 회차의 공연(concertId)은 변경 불가
export class UpdatePerformanceDto extends PartialType(OmitType(CreatePerformanceDto, ['concertId'] as const)) {}
```

○ ○ ○ curl

```
$ curl -X PATCH localhost:4025/performances/1 -d '{"concertId":2}'
{"…","status":400,"errors":["property concertId should not exist"], …}
$ curl -X PATCH localhost:4025/performances/1 -d '{"startsAt":"2026-11-21T19:00:00+09:00"}'
{"id":1,"concertId":1,"startsAt":"2026-11-21T19:00:00+09:00"} [200]
$ curl -X PATCH localhost:4025/performances/1 -d '{"startsAt":"11월 21일"}'
{"…","status":400,"errors":["startsAt must be a valid ISO 8601 date string"], …}
```

© 한상훈 NestJS 백엔드 마스터

매핑된 타입 세 개

이름	만드는 것	예
`PartialType(A)`	A의 필드 전부 선택	공연 수정
`OmitType(A, [키])`	A에서 키를 뺀 것	공연을 바꿀 수 없는 회차 수정
`PickType(A, [키])`	A에서 키만 고른 것	공연장만 바꾸는 요청

→ 규칙은 원본 DTO 한 곳에, 나머지는 파생

→ 겹쳐 쓰기: `PartialType(OmitType(CreatePerformanceDto, ['concertId']))`

→ 패키지: `@nestjs/mapped-types`

데이터 블록: @nestjs/mapped-types 12.0.0 (2026-10-26), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

파이프가 보는 것

파이프에서	파이프 밖에서
금액이 1 이상의 정수인가	이 좌석이 아직 남았는가 (서비스)
좌석 번호가 A-3-15 형식인가	이 좌석이 이 회차에 있는가 (서비스)
날짜가 ISO 8601인가	공연 시작 전인가 (서비스)
받지 않을 필드가 있는가	이 직원이 팔아도 되는가 (가드)

→ 파이프: 값의 모양, 다른 데이터를 조회하지 않음

→ 실행 시점: 가드 뒤 → 가드는 검증 전 값을 봄

→ 서비스의 `typeof amount` 검사: 파이프가 할 일을 서비스가 하는 신호

© 한상훈 NestJS 백엔드 마스터

파이프

"핸들러가 받는 값은 파이프를 통과한 값이고, 통과 조건은 DTO 클래스에 적습니다."